# Client vs Server: Crash vs Shutdown

What each side sees, and the exact error codes, for every combination

## The one idea that decides everything

Whether the **client** or the **server** goes away, the question that determines what error (if any) the *other* side sees is always the same: **did the operating system on the side that went away get a chance to clean up the socket, or did the whole machine disappear at once?**

- If the OS is still alive (even if *your program* crashed) → it cleans up the dead connection for you → looks almost the same as a graceful shutdown.
- If the whole machine/OS is gone → **nothing** gets sent → the other side just hangs until a long timeout.

Everything below is organized as: **what does the SERVER do**, and **what does the CLIENT see as a result** — then the same thing in reverse.

---

## 1. SERVER shuts down normally (calls `close()` then `exit()`)

**What happens:** the server's OS sends a **FIN** to the client on that connection.

**What the CLIENT sees:** - Its next `read()` on the socket returns `0`. - **`0` is not an error** — it's clean end-of-file, meaning "the server is done sending, on purpose." - If the client then tries to `write()` more data to the server: depends on whether the server fully closed (both directions) or just stopped reading — usually you'll eventually get `EPIPE`/`SIGPIPE` (see §5) once the server side is fully gone.

---

## 2. SERVER process crashes (segfault, uncaught exception, `kill -9`) — but the server *machine* is still up

**What happens:** the server's **operating system** is still running, even though your server *program* just died. The OS notices the dead process still had this socket open, and — exactly as if `close()` had been called — sends a **FIN** to the client on its behalf.

**What the CLIENT sees:** - **Usually identical to a graceful shutdown** — `read()` returns `0`. A `kill -9` on your server does **not**, by itself, produce a special "the server crashed" error on the client. This surprises almost everyone the first time. - **Exception — if the server still had unread data sitting in its receive buffer** (data the client sent that the server never got around to `read()`ing before it died): the OS sends a **RST** instead of a FIN. - In that RST case, the client's next `read()` or `write()` gets **`-1`**, with **`errno == ECONNRESET`** ("Connection reset by peer").

---

## 3. SERVER's whole machine dies, or the network link is cut (power loss, VM killed, cable pulled, network partition)

**What happens:** there's no operating system left alive on the server side to send anything at all. **Silence.**

**What the CLIENT sees:** - `read()` just **hangs indefinitely** (unless the client set a read timeout). - `write()` may appear to succeed at first (the data just sits in the client's local send buffer), while TCP quietly retries delivery in the background. Once retransmission attempts are exhausted — this can take **minutes** — the call finally fails with **`-1`**, **`errno == ETIMEDOUT`** ("Connection timed out"). - If the client enabled `SO_KEEPALIVE`, the OS itself will periodically probe the dead connection and eventually fail it with `ETIMEDOUT` even if the client isn't actively reading/writing at that moment.

---

## 4. CLIENT shuts down normally / CLIENT crashes / CLIENT machine dies

**Exactly the same three scenarios as above, mirrored — the SERVER is now the one receiving the signal:**

| Client-side event | What the SERVER sees |
|---|---|
| Client calls `close()`/exits normally | Server's `read()` on that client's socket returns `0` (clean EOF) |
| Client process crashes, client's OS still alive, no pending unread data | Server's `read()` returns `0` — looks the same as a graceful close |
| Client process crashes, client's OS still alive, unread data was pending | Server gets `-1`/`ECONNRESET` on its next read/write to that client |
| Client's whole machine dies / network cut | Server's `read()`/`write()` to that client hangs, then eventually `-1`/`ETIMEDOUT` |

**This matters a lot for your concurrent server design:** each client is handled by its own thread/child process, so one client dying (any of the three ways above) should only affect *that* thread's `read()`/`write()` calls — it should never crash the whole server or block other clients, as long as you check return values properly per-connection.

---

## 5. Writing to a connection where the other side is already gone

Regardless of which side is "yours" (client or server), if you call `write()` on a socket whose peer has already closed or been reset:

- You get **`SIGPIPE`** — default action: **terminates your process** — unless you've caught or ignored it.
- **If you catch/ignore `SIGPIPE`:** the `write()` call itself instead returns **`-1`** with **`errno == EPIPE`**.
- **This is identical, on purpose, to your Signals unit's "write to a pipe with no reader" case** — same mechanism, same fix: install a `SIGPIPE` handler (exactly like your FIFO client-server assignment required) instead of letting the default action silently kill your client or server.

---

## 6. Trying to *connect* to a server that's gone (before any data is even exchanged)

| Server-side condition | What the CLIENT's `connect()` does |
|---|---|
| Server **process** is dead, but its machine/OS is up, and nothing is listening on that port anymore | Fails almost immediately: **`-1`**, **`errno == ECONNREFUSED`** — the server's OS actively sends back a RST because no process owns that port |
| Server's whole **machine** is unreachable (network down, host off) | `connect()` hangs, then eventually fails with **`-1`**, **`errno == ETIMEDOUT`** |

---

## Summary table

| Scenario | What's sent on the wire | What the OTHER side's `read()` gets | What the OTHER side's `write()` gets |
|---|---|---|---|
| Graceful `close()`/`exit()` (either side) | FIN | `0` (EOF — not an error) | works, then `EPIPE`/`SIGPIPE` once fully closed |
| Process crash, OS alive, no pending unread data (either side) | FIN (sent by the OS for you) | `0` (EOF — looks graceful) | same as above |
| Process crash, OS alive, unread data was pending (either side) | RST | `-1` / `ECONNRESET` | `-1` / `ECONNRESET` |
| Whole machine/network dies (either side) | nothing at all | hangs, then `-1` / `ETIMEDOUT` | hangs, then `-1` / `ETIMEDOUT` |
| Client `connect()`s to a dead port (server process gone, machine up) | RST | — | client's `connect()`: `-1` / `ECONNREFUSED` |
| Client `connect()`s to an unreachable server machine | nothing | — | client's `connect()`: hangs, then `-1` / `ETIMEDOUT` |
| Either side writes after the other is already gone | — | — | `-1` / `EPIPE` (+ `SIGPIPE` if not caught) |

---

## The one sentence to remember for an exam

**A clean shutdown and a same-machine crash look identical to the other side — both just show up as `read()` returning 0 — because the OS cleans up a dead process's sockets exactly as if `close()` had been called. Real, distinct errors only appear in two cases: unread data was still sitting in the buffer when a process died (`ECONNRESET`), or the entire machine/network is gone and nothing can respond at all (hang, then `ETIMEDOUT`).**